

Museopica (MIDIソフト風作曲アプリ)

3年9組24号 西川 慎海

1. アプリ仕様

1.1. アプリ概要

本アプリは、MIDIソフト風の作曲アプリです。

白鍵50鍵、黒鍵35鍵、ドラム9鍵の計94鍵からなる鍵盤を備えており、グリッド上にノーツを配置することで、直感的に楽曲制作を行うことが可能です。

音源には外部ライブラリであるsoundfont-playerを採用し、100種類以上の楽器の中から厳選した15種類を使用可能です。また、ドラム音源については、iOS版GarageBandからドラム音のみを抽出し、アプリ内で再生できるようにしました。

拍数の変更、ノーツカラーの変更、テンポ調整、キー変更といった基本的な作曲機能にも対応しています。作曲した楽曲データはJSON形式で保存できますが、現時点では保存したJSONデータの読み込みには対応しておらず、再編集や再生は行えません。

グリッドは1ページにつき4小節を表示し、対応音域はC1～C8までの7オクターブ構成です。各操作ボタンのアイコンにはGoogle Material Iconsを使用し、ブラウザ委のタブに表示されるアイコンはChatGPTの画像生成機能によって制作しました。

各機能の詳細については後述します。

1.2. アプリ構成

このアプリの構成は以下の通りです。

```
Museopica/  
├ assets/  
│   ├── audio/  
│   │   └─ ドラム音源9種(m4a)  
│   ├── img/  
│   │   ├── png/  
│   │   │   └─ icon.png(ブラウザタブ用アイコン)  
│   │   └─ svg/  
│   │       └─ ボタンアイコン11種(svg)  
│   └─ json/  
│       ├── drum.json  
│       ├── inst.json  
│       └─ pitch.json  
├ css/  
│   └─ style.css  
├ js/  
│   └─ main_script.js  
└─ index.html
```

1.3. 使用言語

このアプリに使用した言語は以下の通りです。

- HTML
- CSS
- JavaScript
- JSON(データ保存用)

1.4. セーブデータ(曲データ)仕様

このアプリ専用のセーブデータの構造は以下の通りです。

※可読性を上げるため「”」や「,」などは省略しています。

```
{
  notes: [
    {
      id: ノーツID
      pos: {
        x: ノーツのx座標始点
        y: ノーツのy座標始点
        w: ノーツの幅
        h: ノーツの高さ
      } (ノーツの位置等)
      inst: 楽器名 (soundfont-player準拠)
      inst: キータイプ (白鍵 or 黒鍵 or ドラム)
      pitch: 音階 (MIDIノーツ番号)
      dur: 演奏時間 (s)
      crap: 拍数
      start: 開始時間 / 待ち時間 (s)
      vol: 音量 (ベロシティ)
      color: ノーツカラー
      page: ノーツが設置されているページ
      layer: ノーツが設置されているレイヤー
    }
    {... (設置されたノーツの数のオブジェクトが配列に格納されます) }
  ] (ノーツ1つ1つのデータが保存される配列)
  page_options: [
    {
      tempo: テンポ
      key: 曲のキー
      change_time: ページ切り替えタイミング (s)
      reference: 拍子記号 分母 (基準音符)
      measure: 拍子記号 分子 (1小節当たりの音符数)
      displayed_measure: 表示する小節数
    }
    {... (ページごとのオブジェクトが配列に格納されます) }
  ] (ページごとのデータが保存される配列)
  ver: このアプリのバージョン
}
```

1.5. 作ろうと思ったきっかけ

かねてより、MIDIシステムや作曲に興味があり、「自分も作曲に挑戦してみたい」と思うようになりました。しかし、MIDIソフトやDAWソフトは高価で、十分なお金を持っていなかった僕は、作曲への挑戦を一度諦めることにしました。

そこで僕は、中学生の頃から取り組んでいたScratchを使い、「作曲アプリを自分で開発すればいいのではないか」と考えました。こうして高校1年生の夏から、現在までMIDI風作曲アプリの開発を続けてきました。

そして今回、総合実践の授業で自由にアプリ開発ができる課題が提示されたことをきっかけに、熊商での3年間の学びやAIを最大限活用し、Scratchではなく、1つのWebアプリとして開発しようと考えました。

2. アプリ画面解説

2.1. 画面解説

2.1.1. 上部UI



図1:画面左上 ボタン軍

画面左上にあるボタン解説 (左から順)

ファイルボタン:ファイルの保存や読み込み・設定の変更などができます。

ロックボタン:編集不可モードです。

選択ボタン:設置したノーツの位置やプロパティを変更するモードです。

編集ボタン:ノーツの設置をするモードです。

削除ボタン:ノーツの削除をするモードです。

スキップボタン:曲の1番最初に戻ります。

再生 / 停止ボタン:曲の再生 / 停止をします。



図2:画面中央～右上部 ボタン軍

画面中央～右上部にあるボタン解説 (左から順)

楽器選択プルダウン: 15種類の楽器から選択できます。

拍数変更スライダー: 0.125拍～8拍まで0.125拍区切りで変更できます。

音量変更スライダー: ノーツ単体の音量を0%～200%の範囲で変更できます。

カラーピッカー: ノーツ単体の色を変更できます。

テンポ変更テキストボックス: 曲全体のテンポを変更できます。

キー変更テキストボックス: 曲全体のキーを変更できます。

2.1.2. エディター部分

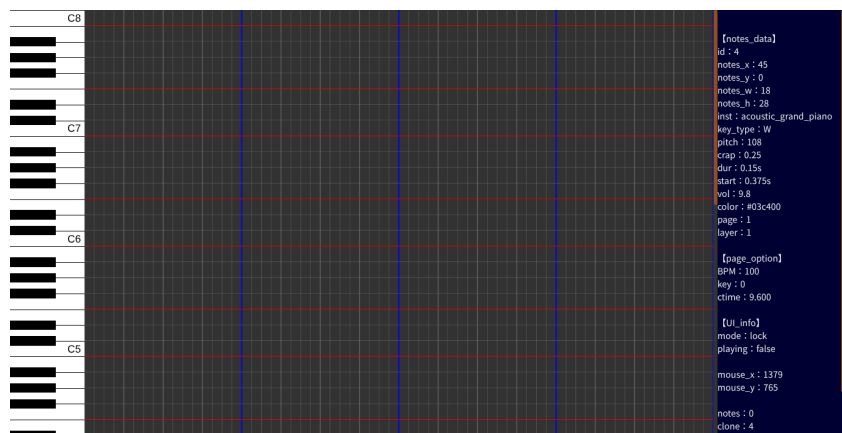


図3:エディター部分

画面左端には音程を示す鍵盤が設置されています。

鍵盤をクリックすることでその音階が再生されます。

画面中央には作曲するグリッドが表示されます。

編集モードに変更し、グリッドをクリックすると、ノーツが設置されます。

画面右端にはデバッグ用の情報が表示されます。

100ms間隔で更新されます。

2.1.3. 下部UI



図4:画面下部 表示

画面下部 表示 (左から順)

ノーツ数表示: 再生済みノーツ数 / 作成した曲の総ノーツ数

再生時間表示: h:mm:ss.000の形式で表示されます。

ページ表示 / ボタン: 現在のページを表示します。オレンジの左右ボタン・テキストボックスでページ移動が可能です。

バージョン表示: このアプリの現在のバージョンを表示します。

3. 各改良ポイント

3.1. HTMLの改良ポイント

3.1.1. DOM要素はできるだけHTMLで

```
<div id="top_ui">
  <button id="file_btn" class="file_btns" title="ファイル">
    
  </button>
  <div id="mode_btns" class="btn_divs">
    <button id="lock_btn" class="mode_btns" data-mode="lock" title="ロックモード">
      
    </button>
    <button id="select_btn" class="mode_btns" data-mode="select" title="選択モード">
      
    </button>
    <button id="create_btn" class="mode_btns" data-mode="create" title="追加モード">
      
    </button>
    <button id="delete_btn" class="mode_btns" data-mode="delete" title="削除モード">
      
    </button>
  </div>
  <div id="ctrl_btns" class="btn_divs">
    <button id="skip_btn" class="ctrl_btns" title="スキップ">
      
    </button>
    <button id="play_btn" class="ctrl_btns" title="再生/停止">
      
    </button>
  </div>
  <select id="inst_sel" class="pulldown" name="instrument" title="楽器選択"><!-- optionはjsで追加 --></select>
  <div id="dur" class="property_divs" title="拍数">
    <label for="dur_input" id="dur_label"><!-- 拍数表記はjsで追加 --></label>
    <input id="dur_input" type="range" min="0.125" max="8" step="0.125" value="0.25" title="拍数">
  </div>
  <div id="vol" class="property_divs" title="音量 (ベロシティ)">
    
    <label for="vol_input" id="vol_label"><!-- 音量 (ベロシティ) 表記はjsで追加 --></label>
    <input id="vol_input" type="range" min="0" max="200" step="5" value="100" title="音量 (ベロシティ)">
  </div>
  <input id="color_input" type="color" value="#03c400" title="ノーツカラー">
  <label id="BPM_label" for="BPM_input">BPM</label>
  <input id="BPM_input" type="number" min="20" max="500" value="100" title="テンポ">
  <div id="BPM_ctrls" class="ctrls">
    <button id="BPM_+" class="BPM_ctrls">+</button>
    <button id="BPM_-" class="BPM_ctrls">-</button>
  </div>
  <label id="key_label" for="key_input">key</label>
  <input id="key_input" type="number" min="-11" max="11" value="0" title="キー">
  <div id="key_ctrls" class="ctrls">
    <button id="key_+" class="key_ctrls">+</button>
    <button id="key_-" class="key_ctrls">-</button>
  </div>
</div>
```

図5:「図1, 2『上部UI』」のHTML

HTMLの工夫点として、今まではHTMLに<div>だけ書いて、それ以外のDOM要素はJavaScriptで記述していました。

その方法だと、アプリのパフォーマンスが落ちてしまうため、鍵盤以外のDOM要素はすべてJavaScriptで記述するように心がけました。

3.2.1. 各機能ごとに処理を分割

3.2.1. 各機能ごとに処理を分割

```

1 const initAudioContext = () => {
2   const audioContext = new AudioContext();
3   let pitchData;
4   let initData = { instruments: [], drum: {} };
5   let playing = false;
6   let measure = 0;
7   let mode;
8   let currentInstrument, keyType, pitch, crap, duration, vol, color, tempo = 100;
9   let notes = 0;
10  let cTime = 0;
11  let playedNotes = 0;
12  let time = 0.00:00.00 // hour, minute, second, key = 1, layer = 1;
13  let start;
14  let startTimeBar, startTimer, elapsedTimeBar, elapsedTime;
15  const notesData = [];
16  const delayed = [];
17  const pageOptions = Array.from({ length: 100 }, () => {});
18  tempo = 100;
19  key = 9;
20  changeTime = 0;
21  reference = 4;
22  measure = 4;
23  displayedMeasure = 4;
24  });
25  const changed = new Array(pageOptions.length).fill(false);
26  tempoChange();
27  let key, vol, crap;
28  const mKeys = new Array(50); // white key points (白鍵の座標 配列)
29  const mKeys = new Array(50); // black key points (黒鍵の座標 配列)
30  const mKeys = new Array(50); // mouse black key points (黒鍵のマウス座標 配列)
31  const mKeys = new Array(50); // drum key points (ドラムの座標 配列)
32  const mKeys = new Array(50); // mouse drum key points (ドラムマウスの座標 配列)
33  const mKeys = new Array(120); // musical notes (音符 配列)
34  const mKeys = new Array(120); // mouse points (マウスの座標 配列)
35  document.addEventListener("DOMContentLoaded", main);
36  //
37  //
38  //
39  //
40  //
41  //
42  //
43  //
44  //
45  //
46  //
47  //
48  //
49  //
50  //
51  //
52  //
53  //
54  //
55  //
56  //
57  //
58  //
59  //
60  //
61  //
62  //
63  //
64  //
65  //
66  //
67  //
68  //
69  //
70  //
71  //
72  //
73  //
74  //
75  //
76  //
77  //
78  //
79  //
80  //
81  //
82  //
83  //
84  //
85  //
86  //
87  //
88  //
89  //
90  //
91  //
92  //
93  //
94  //
95  //
96  //
97  //
98  //
99  //
100 //
101 //
102 //
103 //
104 //
105 //
106 //
107 //
108 //
109 //
110 //
111 //
112 //
113 //
114 //
115 //
116 //
117 //
118 //
119 //
120 //
121 //
122 //
123 //
124 //
125 //
126 //
127 //
128 //
129 //
130 //
131 //
132 //
133 //
134 //
135 //
136 //
137 //
138 //
139 //
140 //
141 //
142 //
143 //
144 //
145 //
146 //
147 //
148 //
149 //
150 //
151 //
152 //
153 //
154 //
155 //
156 //
157 //
158 //
159 //
160 //
161 //
162 //
163 //
164 //
165 //
166 //
167 //
168 //
169 //
170 //
171 //
172 //
173 //
174 //
175 //
176 //
177 //
178 //
179 //
180 //
181 //
182 //
183 //
184 //
185 //
186 //
187 //
188 //
189 //
190 //
191 //
192 //
193 //
194 //
195 //
196 //
197 //
198 //
199 //
200 //
201 //
202 //
203 //
204 //
205 //
206 //
207 //
208 //
209 //
210 //
211 //
212 //
213 //
214 //
215 //
216 //
217 //
218 //
219 //
220 //
221 //
222 //
223 //
224 //
225 //
226 //
227 //
228 //
229 //
230 //
231 //
232 //
233 //
234 //
235 //
236 //
237 //
238 //
239 //
240 //
241 //
242 //
243 //
244 //
245 //
246 //
247 //
248 //
249 //
250 //
251 //
252 //
253 //
254 //
255 //
256 //
257 //
258 //
259 //
260 //
261 //
262 //
263 //
264 //
265 //
266 //
267 //
268 //
269 //
270 //
271 //
272 //
273 //
274 //
275 //
276 //
277 //
278 //
279 //
280 //
281 //
282 //
283 //
284 //
285 //
286 //
287 //
288 //
289 //
290 //
291 //
292 //
293 //
294 //
295 //
296 //
297 //
298 //
299 //
300 //
301 //
302 //
303 //
304 //
305 //
306 //
307 //
308 //
309 //
310 //
311 //
312 //
313 //
314 //
315 //
316 //
317 //
318 //
319 //
320 //
321 //
322 //
323 //
324 //
325 //
326 //
327 //
328 //
329 //
330 //
331 //
332 //
333 //
334 //
335 //
336 //
337 //
338 //
339 //
340 //
341 //
342 //
343 //
344 //
345 //
346 //
347 //
348 //
349 //
350 //
351 //
352 //
353 //
354 //
355 //
356 //
357 //
358 //
359 //
360 //
361 //
362 //
363 //
364 //
365 //
366 //
367 //
368 //
369 //
370 //
371 //
372 //
373 //
374 //
375 //
376 //
377 //
378 //
379 //
380 //
381 //
382 //
383 //
384 //
385 //
386 //
387 //
388 //
389 //
390 //
391 //
392 //
393 //
394 //
395 //
396 //
397 //
398 //
399 //
400 //
401 //
402 //
403 //
404 //
405 //
406 //
407 //
408 //
409 //
410 //
411 //
412 //
413 //
414 //
415 //
416 //
417 //
418 //
419 //
420 //
421 //
422 //
423 //
424 //
425 //
426 //
427 //
428 //
429 //
430 //
431 //
432 //
433 //
434 //
435 //
436 //
437 //
438 //
439 //
440 //
441 //
442 //
443 //
444 //
445 //
446 //
447 //
448 //
449 //
450 //
451 //
452 //
453 //
454 //
455 //
456 //
457 //
458 //
459 //
460 //
461 //
462 //
463 //
464 //
465 //
466 //
467 //
468 //
469 //
470 //
471 //
472 //
473 //
474 //
475 //
476 //
477 //
478 //
479 //
480 //
481 //
482 //
483 //
484 //
485 //
486 //
487 //
488 //
489 //
490 //
491 //
492 //
493 //
494 //
495 //
496 //
497 //
498 //
499 //
500 //
501 //
502 //
503 //
504 //
505 //
506 //
507 //
508 //
509 //
510 //
511 //
512 //
513 //
514 //
515 //
516 //
517 //
518 //
519 //
520 //
521 //
522 //
523 //
524 //
525 //
526 //
527 //
528 //
529 //
530 //
531 //
532 //
533 //
534 //
535 //
536 //
537 //
538 //
539 //
540 //
541 //
542 //
543 //
544 //
545 //
546 //
547 //
548 //
549 //
550 //
551 //
552 //
553 //
554 //
555 //
556 //
557 //
558 //
559 //
560 //
561 //
562 //
563 //
564 //
565 //
566 //
567 //
568 //
569 //
570 //
571 //
572 //
573 //
574 //
575 //
576 //
577 //
578 //
579 //
580 //
581 //
582 //
583 //
584 //
585 //
586 //
587 //
588 //
589 //
590 //
591 //
592 //
593 //
594 //
595 //
596 //
597 //
598 //
599 //
600 //
601 //
602 //
603 //
604 //
605 //
606 //
607 //
608 //
609 //
610 //
611 //
612 //
613 //
614 //
615 //
616 //
617 //
618 //
619 //
620 //
621 //
622 //
623 //
624 //
625 //
626 //
627 //
628 //
629 //
630 //
631 //
632 //
633 //
634 //
635 //
636 //
637 //
638 //
639 //
640 //
641 //
642 //
643 //
644 //
645 //
646 //
647 //
648 //
649 //
650 //
651 //
652 //
653 //
654 //
655 //
656 //
657 //
658 //
659 //
660 //
661 //
662 //
663 //
664 //
665 //
666 //
667 //
668 //
669 //
670 //
671 //
672 //
673 //
674 //
675 //
676 //
677 //
678 //
679 //
680 //
681 //
682 //
683 //
684 //
685 //
686 //
687 //
688 //
689 //
690 //
691 //
692 //
693 //
694 //
695 //
696 //
697 //
698 //
699 //
700 //
701 //
702 //
703 //
704 //
705 //
706 //
707 //
708 //
709 //
710 //
711 //
712 //
713 //
714 //
715 //
716 //
717 //
718 //
719 //
720 //
721 //
722 //
723 //
724 //
725 //
726 //
727 //
728 //
729 //
730 //
731 //
732 //
733 //
734 //
735 //
736 //
737 //
738 //
739 //
740 //
741 //
742 //
743 //
744 //
745 //
746 //
747 //

```

 6: main_script.js

図6の通り、グローバルに定義されている関数は25個あります。

以下は各functionの機能別の一部です

- `async function main()`:すべての関数を実行
- `function keys_generation()`:各鍵盤の生成処理
- `async function top_UI_prepare()`:上部UIボタン等の生成・EventListener登録
- `function play_music()`:一時保存変数(`notes_data`)に基づき、各ノーツの発音

3.3. CSSの改良ポイント

3.3.1. 冗長な処理の最適化

```
#black_keyboards .black_key:nth-child(1) {
    margin-top: calc((var(--key-wh) * 2) - (var(--key-bh) / 2));
}

#black_keyboards .black_key:nth-child(2),
#black_keyboards .black_key:nth-child(3),
#black_keyboards .black_key:nth-child(5),
#black_keyboards .black_key:nth-child(7),
#black_keyboards .black_key:nth-child(8),
#black_keyboards .black_key:nth-child(10),
#black_keyboards .black_key:nth-child(12),
#black_keyboards .black_key:nth-child(13),
#black_keyboards .black_key:nth-child(15),
#black_keyboards .black_key:nth-child(17),
#black_keyboards .black_key:nth-child(18),
#black_keyboards .black_key:nth-child(20),
#black_keyboards .black_key:nth-child(22),
#black_keyboards .black_key:nth-child(23),
#black_keyboards .black_key:nth-child(25) {
    margin-top: calc(var(--key-wh) - var(--key-bh));
}

#black_keyboards .black_key:nth-child(4),
#black_keyboards .black_key:nth-child(6),
#black_keyboards .black_key:nth-child(9),
#black_keyboards .black_key:nth-child(11),
#black_keyboards .black_key:nth-child(14),
#black_keyboards .black_key:nth-child(16),
#black_keyboards .black_key:nth-child(19),
#black_keyboards .black_key:nth-child(21),
#black_keyboards .black_key:nth-child(24) {
    margin-top: calc((var(--key-wh) * 2) - var(--key-bh));
}
```

図7:脳筋CSS

図7は黒鍵の間隔を空けるための処理です。

今の状態だと

```
#black_keyboards .black_key:nth-child(n)
```

が多すぎてとても見づらいCSSになっています。

```
#black_keyboards .black_key:nth-child(1) {
    margin-top: calc((var(--key-wh) * 2) - (var(--key-bh) / 2));
}

#black_keyboards .black_key:nth-child(5n),
#black_keyboards .black_key:nth-child(5n+2),
#black_keyboards .black_key:nth-child(5n+3) {
    margin-top: calc(var(--key-wh) - var(--key-bh));
}

#black_keyboards .black_key:nth-child(5n+1):not(:nth-child(1)),
#black_keyboards .black_key:nth-child(5n+4) {
    margin-top: calc((var(--key-wh) * 2) - var(--key-bh));
}
```

図8:黒鍵 間隔空け処理(最適化後)

ですが、図7の処理には法則性があるため、図8のように最適化することで、コードの可読性を向上させました。

4. 現在の問題と本アプリの将来

4.1. 現在の問題

4.1.1. 画面リサイズ時

画面がフルスクリーン化またはリサイズされた時 (`addEventListener('resize')`) にノーツの再生はされますが、ノーツの描画が消えるまたは位置ズレを起こすという問題があります。最低限作曲ができて、画面の位置やサイズを変えなければバグは起きないので、現在は修正してません。
今後修正する予定です。

4.2. これから追加する機能

4.2.1. 読み込み

現在、作曲データをJSONファイル形式で保存はできますが、読み込みには対応していないため、ファイルの再編集やファイルを読み込んで再生ができません。作曲アプリとして読み込みができないと意味がないので、本レポートが書き終わり次第すぐにv1.1として実装し、公開します。

4.2.2. selectモード

現在、ノーツの追加と削除はできますが、設置されたノーツの再編集(ノーツ位置、音階、楽器、拍数、音量、色など)は出来ません。ノーツのプロパティを変更したい場合は削除してから置き直さないといけないというのはとても手間になるので、早めに実装したいです。

4.2.3. レイヤー機能

この機能はグリッドにノーツで溢れて大変なことになるため、レイヤー機能を実装することで見やすい状態で作曲できるようになると思います。

4.2.4. その他

その他は「これ、あったら便利だなあ」と思えるような機能です。

以下は機能例です。

- ページごとにテンポ・キーを変更可能に
- 4/4以外の拍子の実装(3/4や6/8など・ページごとに変更可)
- 3連符に対応(1/3拍や2/3拍)

5. 実行する際の注意点

5.1. 実行デバイスについて

このアプリは一般的なWebアプリと比較して、かなりの量のメモリを消費します。(2GB以上)

そのため、スマホでは落ちてしまうため、実行しないことをおすすめします。

推奨デバイスはメモリを4GB以上搭載したPCまたはiPadなどのタブレット、学校のChrome Bookなどです。

また、インターネット回線が遅いと、最初の楽器読み込みに時間がかかります。

ずっとローディング画面が出続けますがリロードはせず、再読み込みをする場合は、一度タブを閉じてから再読み込みをしてください。

5.2. 作曲に関する注意

「4.1. 現在の問題」でも記述した通り、フルスクリーンなど画面サイズを変更するとノーツの描画が消えるまたは位置ズレを起こすという問題があります。

そのため、ノーツを設置したあとは画面サイズを変更しないようにしてください。

ノーツの再生には影響ありません。

6. 作曲アプリの開発を通しての感想

「1.5. 作ろうと思ったきっかけ」でも述べた通り、Scratch版の作曲アプリは「4.2. これから追加する機能」で書いた機能をほぼ全て実装しており、2年以上開発してきました。

そんなアプリを約1ヶ月間で完璧に再現するのは至難の業だったため、たくさんの機能を削ぎ落とし最低限作曲が可能な範囲まで作成しました。

今までScratchで作ったロジックと全く違う処理方法で作成したかつ、熊デパアプリでは書かない新しい書き方もたくさんあったため、とても勉強になりました。

これからもこの作曲アプリは進化を続けます。楽しみにしていてください。

7. 1年間の振り返り

この1年間の総合実践での活動を振り返ってみると、課題研究アプリ開発講座で同じ言語をやっていたが故にかなり勉強になりました。

ですが、未だに

```
for (let i = 0; i >= 10; i++)
```

のように不等号の向きを間違えてたり、10回繰り返したいはずなのに11回繰り返すように書いていたりします。

これからはこのような小さなミスや混乱をなくしていけるように頑張ります。

